

Maestría en Software

Autora: Ing. Patsy Malena Prieto MSc.



| POSGRADOS |

Bienvenida

UNIDAD 2 -

Tema: Arquitectura y Patrones para sistemas dirigidos por APIs

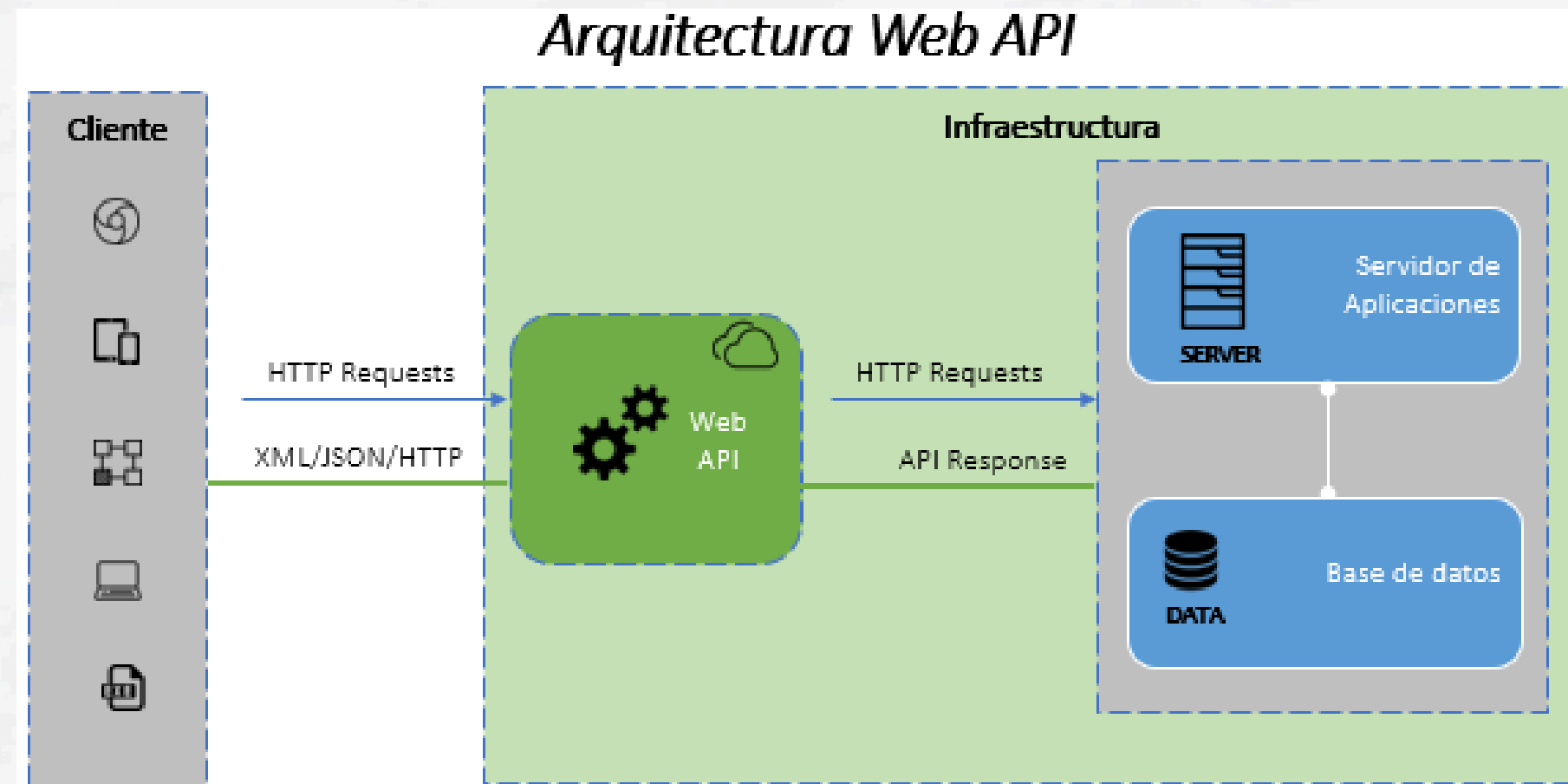
- Arquitecturas de software orientados a APIs
- Patrones de diseño de software orientados a APIs

Arquitecturas de software orientadas a APIs

Definición

La arquitectura de software es un proceso fundamental en la creación de sistemas que va más allá de un plan estático; es un "viaje sin destino". Esto significa que una buena arquitectura debe permitir que un sistema cambie y se adapte incrementalmente a lo largo del tiempo para responder a nuevas tecnologías y requisitos del negocio.

Una definición clave es que la arquitectura de software se centra en aquellas decisiones que son importantes y difíciles de cambiar

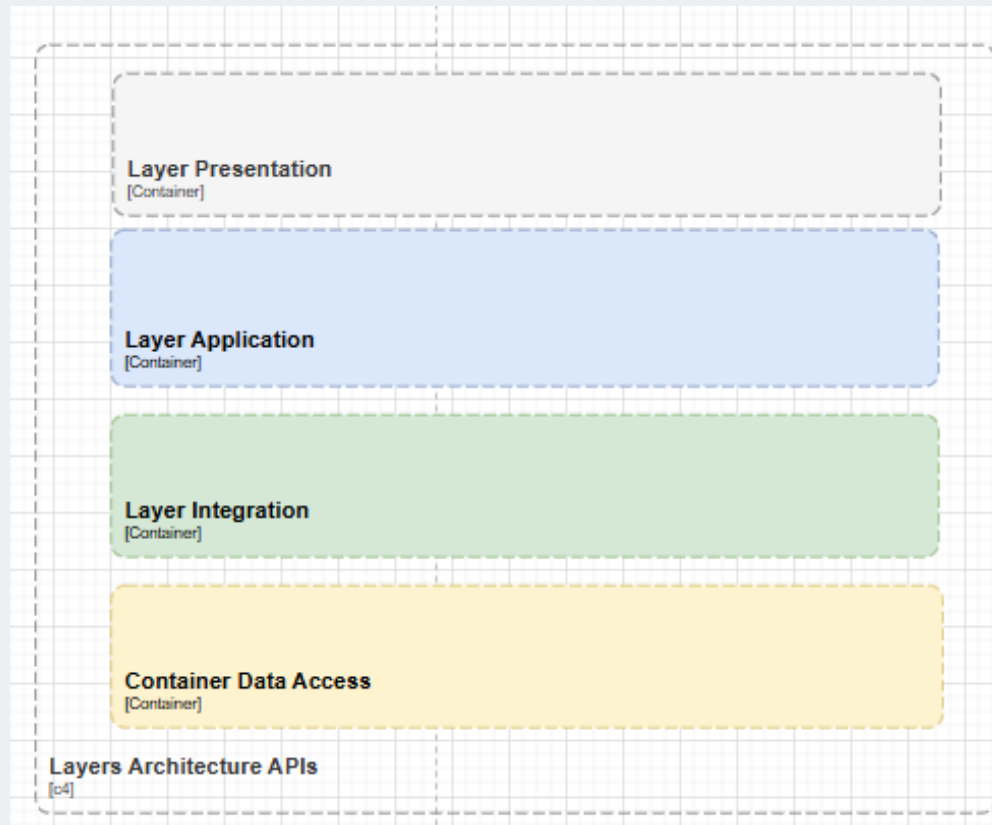


Una buena arquitectura promueve el bajo acoplamiento (los componentes no dependen estrechamente unos de otros) y la alta cohesión (los elementos dentro de un componente están relacionados lógicamente). Estos principios hacen que los sistemas sean más fáciles de entender, mantener y modificar

Tipos de estilos arquitectónicos

- Monolítica
- Arquitectura orientada a servicios SOA
- REST
- Microservicios
- Event Driven
- gRPC (Google Remote Procedure Call)
- Serverless / FaaS expuesto vía APIs

Arquitectura de Software Orientada a APIs



Capa de Presentación (Interfaz de Usuario)

Es la capa que interactúa directamente con el usuario, mostrando la información y recopilando los comandos del cliente.

Capa de Aplicación

Alberga la lógica de negocio y define el funcionamiento de la API. Procesa solicitudes, ejecuta operaciones y orquesta el comportamiento general de la API. Es el núcleo funcional de la arquitectura de la API.

Capa de integración

Facilita la interoperabilidad gestionando la integración entre sistemas y servicios. Realiza tareas como la transformación de datos y validación, asegurando un flujo continuo de información. Se ubica entre la capa de datos y la capa de aplicación.

Capa de Acceso a Datos

Responsable de interactuar con el almacenamiento de datos, realizando operaciones como crear, leer, actualizar y eliminar (CRUD).

Tipos de arquitecturas de software orientadas a APIs

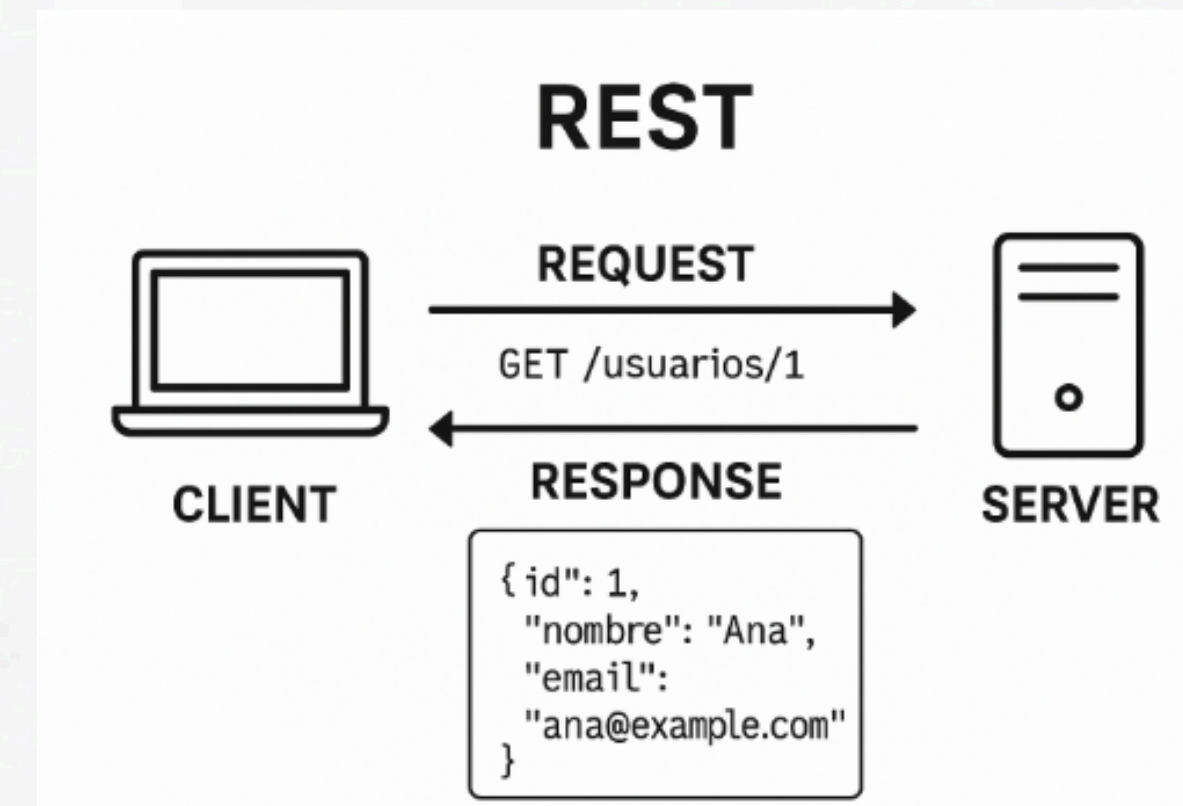
Monolítico expuesto por APIs

- Una aplicación única, que expone su funcionalidad vía un API REST/GraphQL.
- Ventaja: Simplicidad inicial, un solo despliegue.
- Limitación: Escalabilidad y evolución limitadas; cambios en el API afectan todo el monolito.



RESTful

- Usa HTTP como protocolo principal.
- Los recursos (ej.: usuarios, productos, pedidos) se representan en formato JSON (habitualmente).
- Sigue el patrón URI = recurso.
- Es independiente de la plataforma (puede ser consumida desde web, móvil, IoT).



Tipos de arquitecturas de software orientadas a APIs

Microservicios dirigidos por APIs

- Cada servicio es autónomo con su propia base de datos y API.
- El API Gateway centraliza la entrada.
- Ventaja: Escalabilidad, resiliencia, despliegues independientes.
- Limitación: Complejidad operativa (observabilidad, fallos distribuidos, seguridad).



Event-Driven APIs

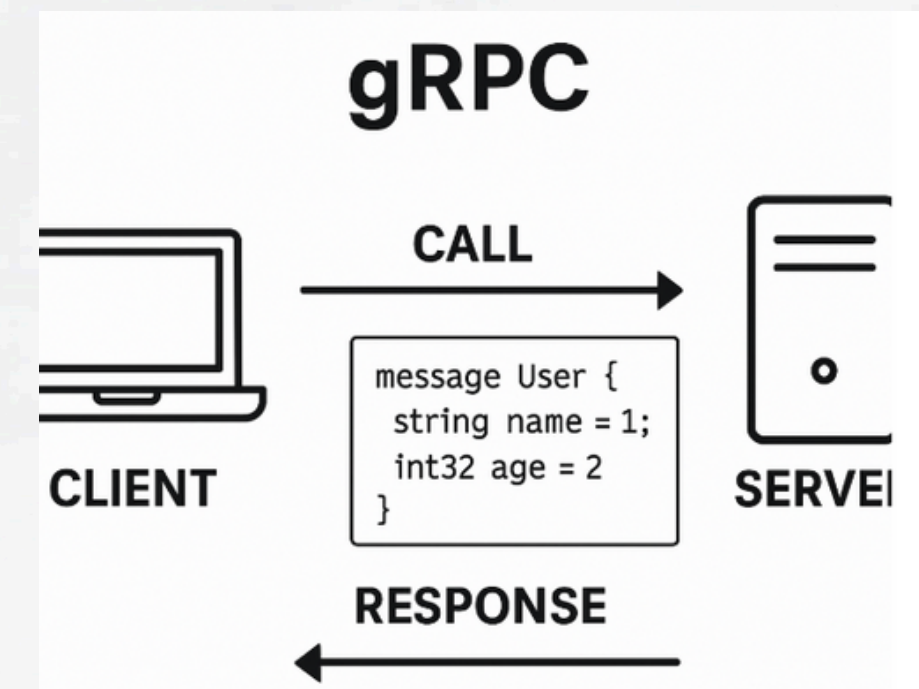
- Los servicios publican y consumen eventos (Kafka, RabbitMQ, Webhooks, SSE, gRPC streaming).
- Ventaja: Desacoplamiento, escalabilidad, tiempo real.
- Limitación: Complejidad en consistencia eventual y orden de eventos.



Tipos de arquitecturas de software orientadas a APIs

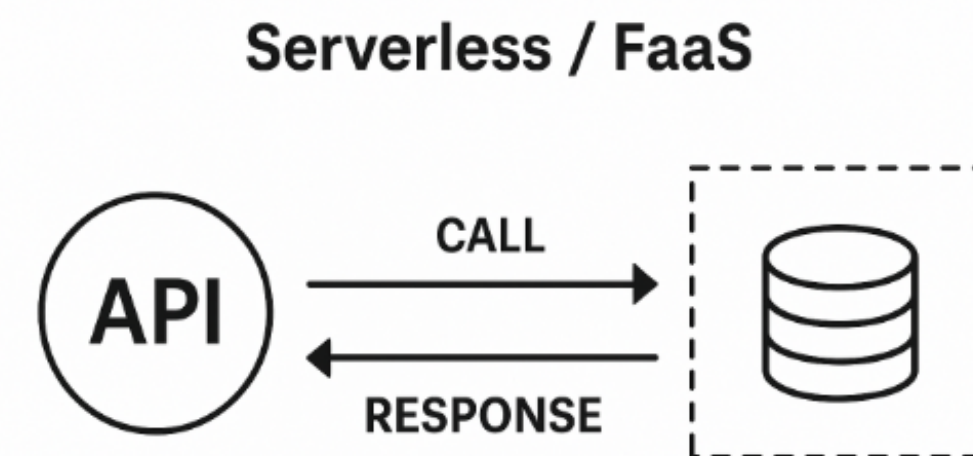
gRPC (Google Remote Procedure Call)

- Una aplicación única, que expone su funcionalidad vía un API REST/GraphQL.
- Ventaja: Simplicidad inicial, un solo despliegue.
- Limitación: Escalabilidad y evolución limitadas; cambios en el API afectan todo el monolito.



Serverless / FaaS expuesto vía APIs

- Una capa intermedia que adapta las APIs según el tipo de cliente (web, móvil, IoT).
- Ventaja: Experiencia optimizada por dispositivo; clientes más simples.
- Limitación: Multiplicación de backends que mantener.

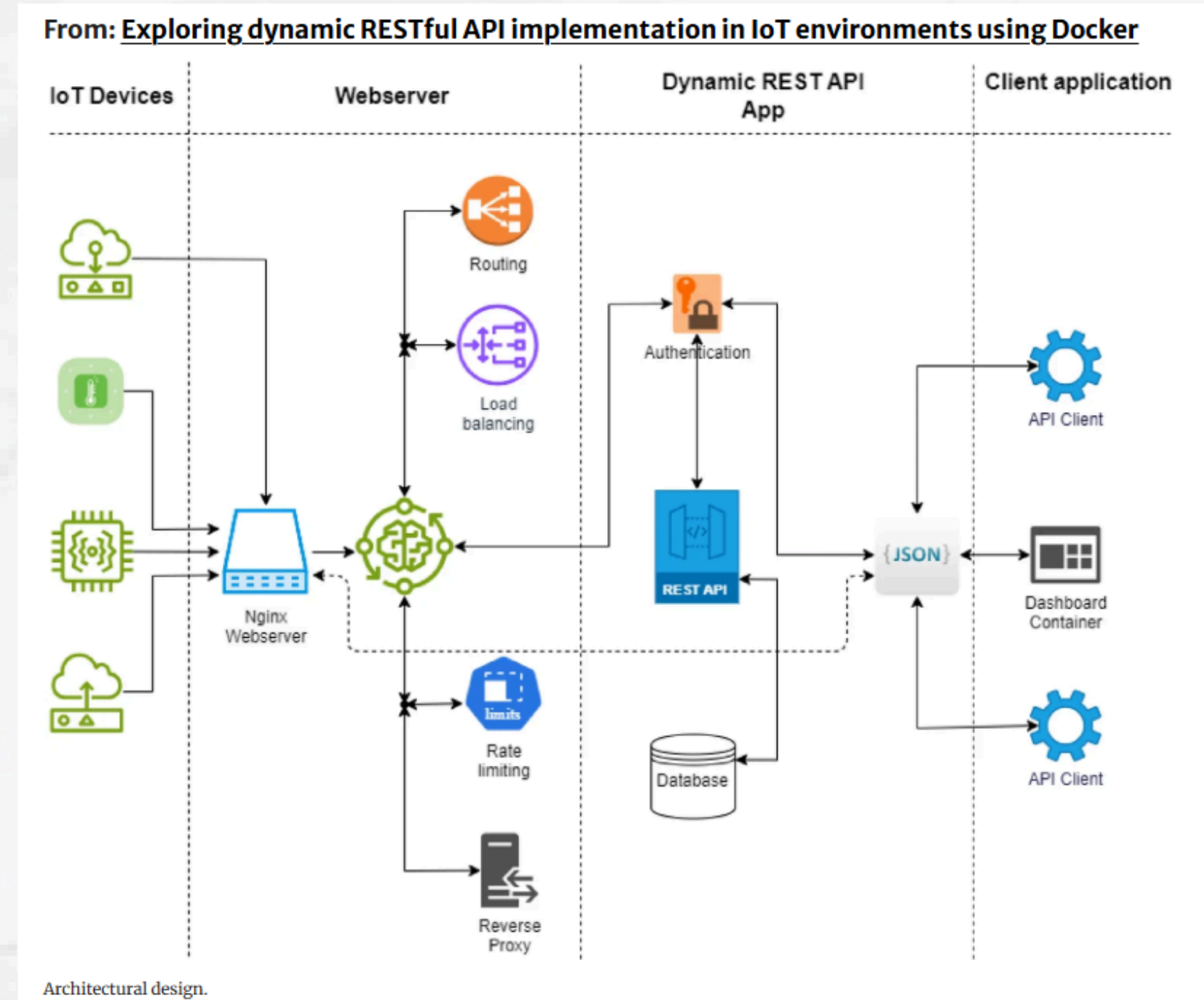


REST (Representational State Transfer)

Abstract

Effective deployment solutions are essential for maximizing the capabilities of Internet of Things (IoT) devices and platforms. This study proposes a technique for enhancing the management, monitoring, and deployment of Internet of Things (IoT) devices, focusing on Dynamic RESTful APIs and Docker technologies. The suggested framework emphasizes reliable interaction and real-time flexibility between IoT devices and deployment infrastructures via a Dynamic RESTful API, combined with the deployment convenience given by Docker's lightweight containerization.

Mabotha, E., Mabunda, N.E., Ali, A. et al. Exploring dynamic RESTful API implementation in IoT environments using Docker. Sci Rep 15, 34267 (2025). <https://doi-org.ecups.idm.oclc.org/10.1038/s41598-025-16460-0>.



REST (Representational State Transfer)

✨ Principios básicos de REST

1. Cliente-Servidor

- El cliente (navegador, app móvil) pide recursos.
- El servidor responde con representaciones de esos recursos.
- Desacopla interfaz de usuario y almacenamiento de datos.

2. Sin estado (Stateless)

- Cada petición HTTP contiene toda la información necesaria.
- El servidor **no guarda sesión** entre llamadas.

3. Cacheable

- Las respuestas deben indicar si se pueden cachear (`Cache-Control` , `ETag`).
- Mejora rendimiento y reduce carga en el servidor.

4. Interfaz uniforme (Uniform Interface)

- Recursos identificados con **URLs**.
- Se manipulan con **métodos HTTP estándar** (GET, POST, PUT, DELETE).
- Representación común: JSON, XML, YAML, etc.

REST (Representational State Transfer)

5. Sistema en capas

- El cliente no necesita saber si habla directo con el servidor o con un proxy/gateway.

6. Código bajo demanda (opcional)

- El servidor puede enviar código ejecutable al cliente (ej.: JavaScript).

🔑 Características de una API RESTful

- Usa **HTTP** como protocolo principal.
- Los **recursos** (ej.: usuarios, productos, pedidos) se representan en formato **JSON** (habitualmente).
- Sigue el patrón **URI = recurso**.
- Es **independiente de la plataforma** (puede ser consumida desde web, móvil, IoT).

REST (Representational State Transfer)

Métodos HTTP en REST

Método	Uso	Ejemplo
GET	Leer recurso	<code>GET /usuarios/1</code>
POST	Crear recurso	<code>POST /usuarios</code>
PUT	Reemplazar recurso	<code>PUT /usuarios/1</code>
PATCH	Modificar parcialmente	<code>PATCH /usuarios/1</code>
DELETE	Eliminar recurso	<code>DELETE /usuarios/1</code>

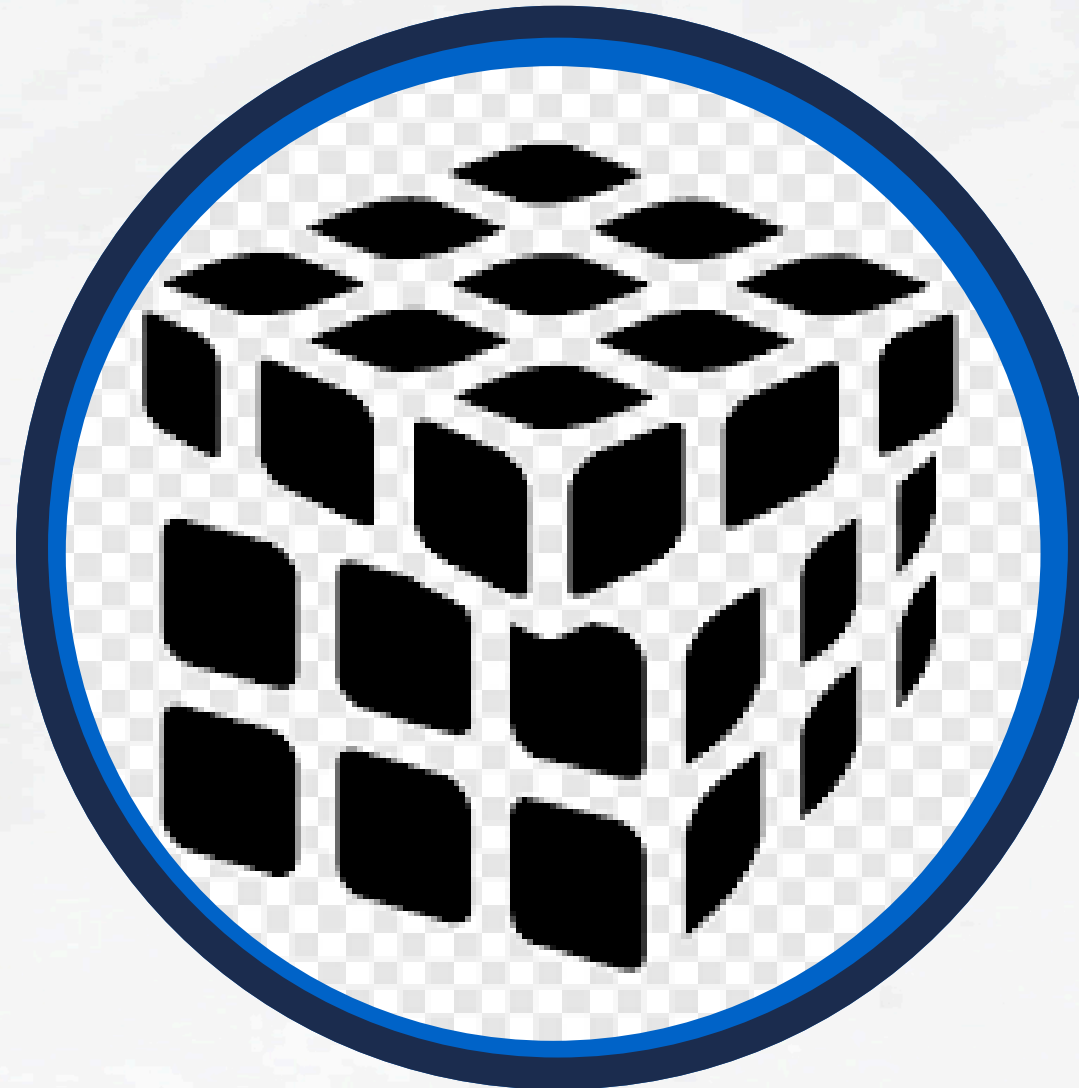
Patrones de diseño de software orientados a APIs

¿Qué es un patrón de diseño?

Los patrones de diseño de APIs son soluciones reutilizables para problemas comunes en la construcción, publicación y consumo de APIs. Establecen buenas prácticas que mejoran la consistencia, la escalabilidad, la seguridad y la experiencia del consumidor. Se aplican tanto a nivel de arquitectura como de implementación.

Los patrones de diseño son un juego de herramientas de soluciones comprobadas a problemas habituales en el diseño de software.

MSOF- PATRONES DE DISEÑO DE APIS



Los patrones de diseño de APIs permiten resolver problemas recurrentes de integración y consumo, ayudando a construir sistemas más robustos, seguros y fáciles de mantener.

Usar patrones en el diseño de APIs ahorra tiempo, reduce riesgos y asegura calidad. Sin patrones, cada equipo tendería a resolver problemas de forma distinta, generando inconsistencia, inseguridad y dificultad de mantenimiento.

Tipos de patrones

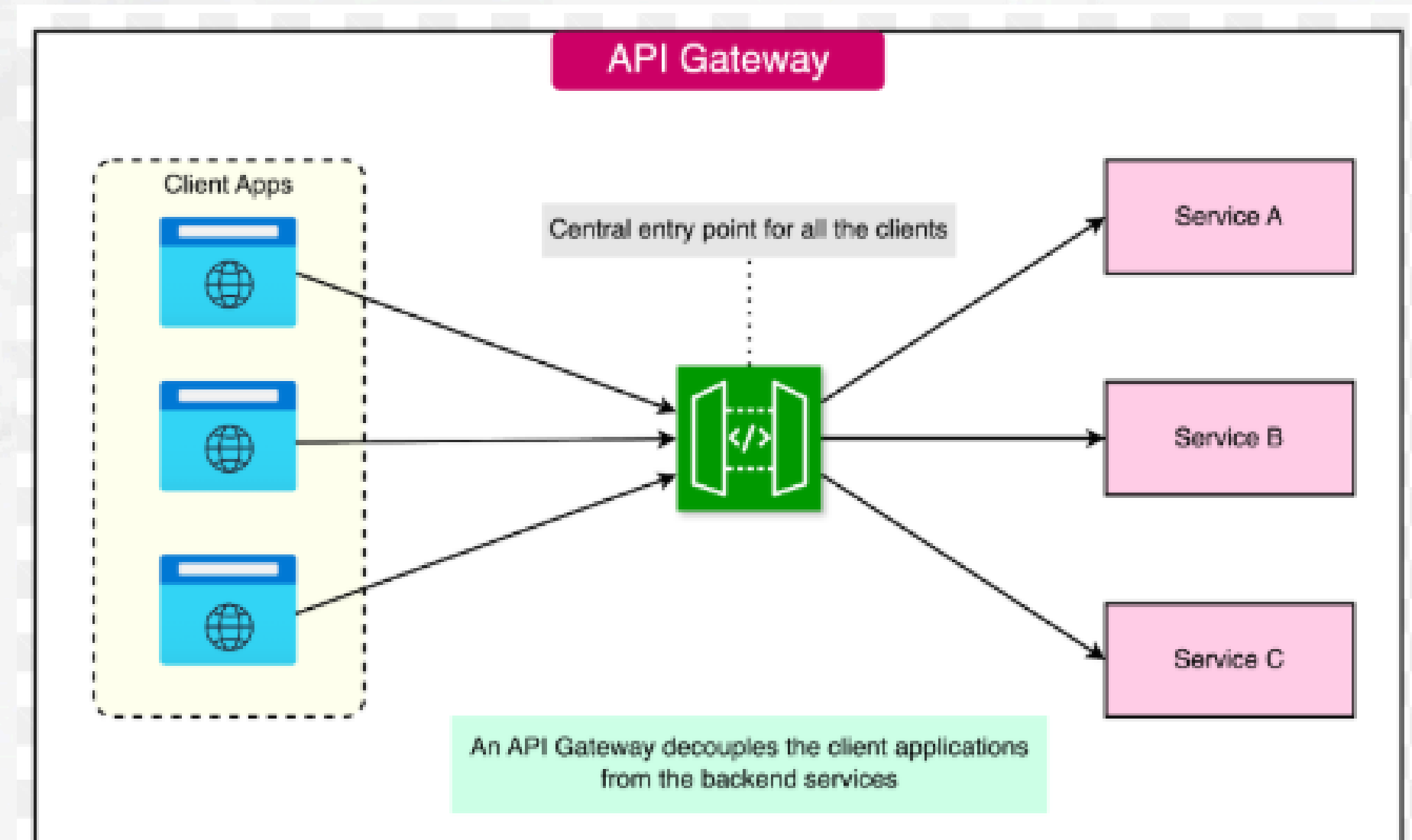
Patrón API Gateway
Patrón Service Registry
Patrón BFF Backend for Frontend
CQRS (Command Query Responsibility Segregation)
Patrón Circuit Breaker

Tipos de patrones de software orientados a APIs

Patrón API Gateway

Un API gateway es una herramienta de gestión crítica que se sitúa en el "borde" (edge) de un sistema. Funciona como un único punto de entrada que media entre los consumidores (como aplicaciones móviles, navegadores web o sistemas de terceros) y una colección de servicios de backend.

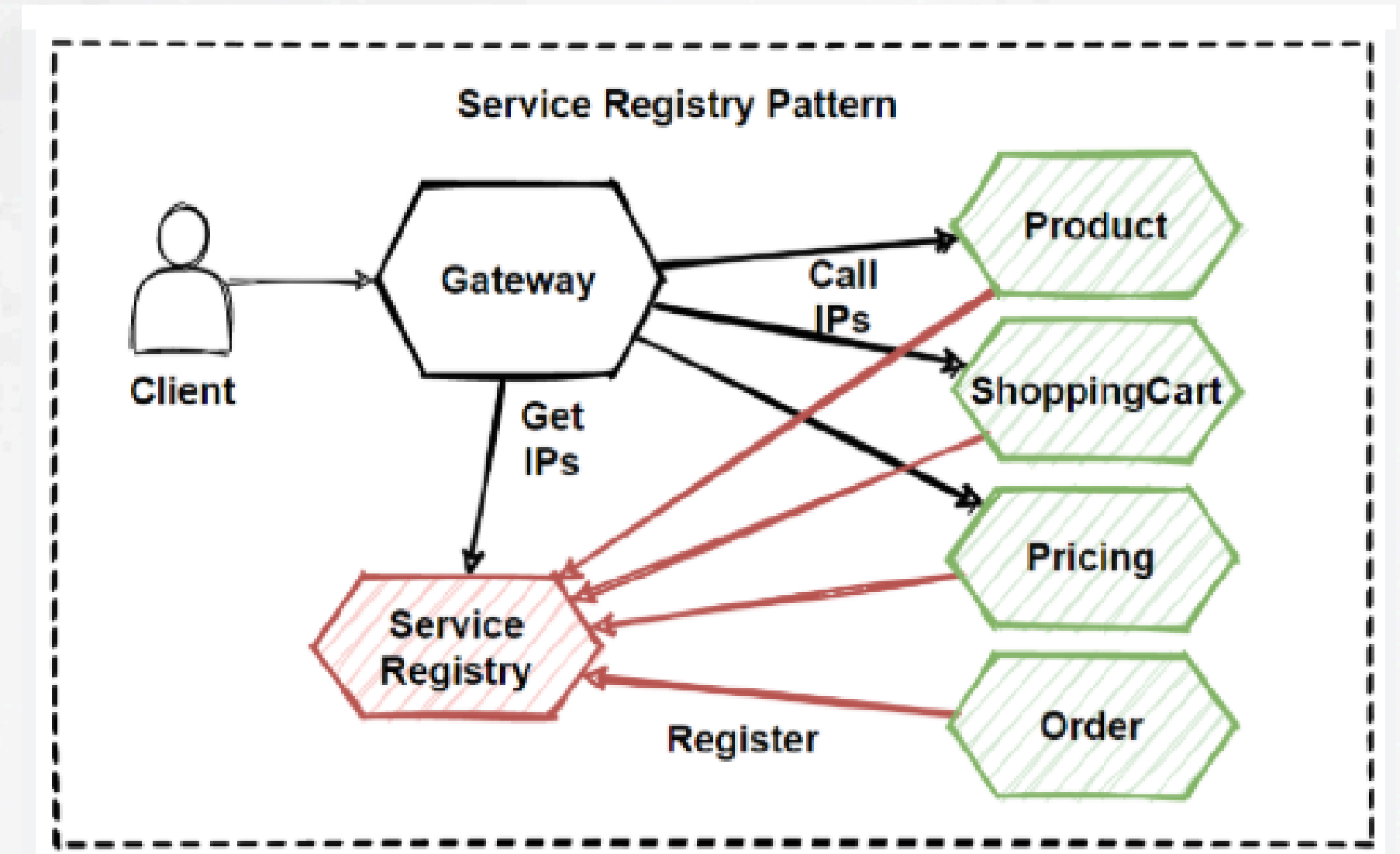
Un API gateway actúa como un proxy inverso que acepta todas las solicitudes de API de un consumidor, las dirige a los servicios de backend apropiados, agrega los resultados y devuelve la respuesta adecuada



Tipos de patrones de software orientados a APIs

Patrón Service Registry

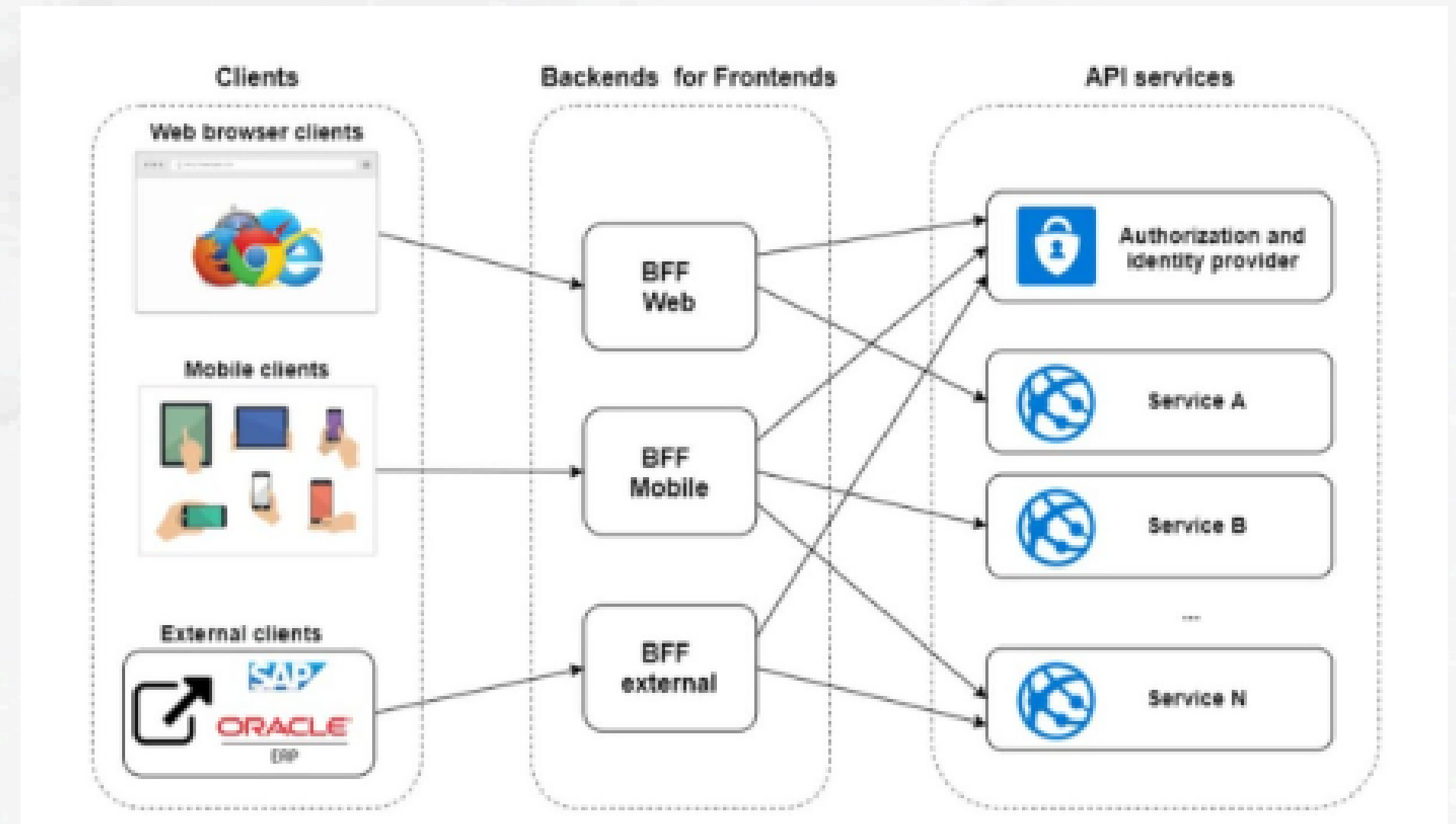
es un patrón de diseño fundamental en arquitecturas de microservicios que permite el descubrimiento dinámico de servicios y reduce la dependencia de configuraciones estáticas. Un registro de servicios centralizado actúa como un catálogo donde los microservicios se registran al iniciarse, proporcionando su ubicación (dirección IP y puerto). Cuando un servicio necesita comunicarse con otro, consulta este registro para encontrar la dirección y el estado de las instancias disponibles, asegurando un desacoplamiento y una mayor resiliencia en sistemas escalables y dinámicos.



Tipos de patrones de software orientados a APIs

Backend for Frontend

- Es un enfoque arquitectónico que consiste en crear un servicio de backend dedicado para cada cliente frontend (como una aplicación web, móvil o de escritorio) en lugar de tener una API monolítica que sirva a todos ellos. Este patrón actúa como una capa intermedia que adapta y filtra los datos de los servicios backend centrales para satisfacer las necesidades específicas de cada interfaz, optimizando la experiencia del usuario, la agilidad del desarrollo y el rendimiento del sistema.
- Ventaja: Experiencia optimizada por dispositivo; clientes más simples.
- Limitación: Multiplicación de backends que mantener.

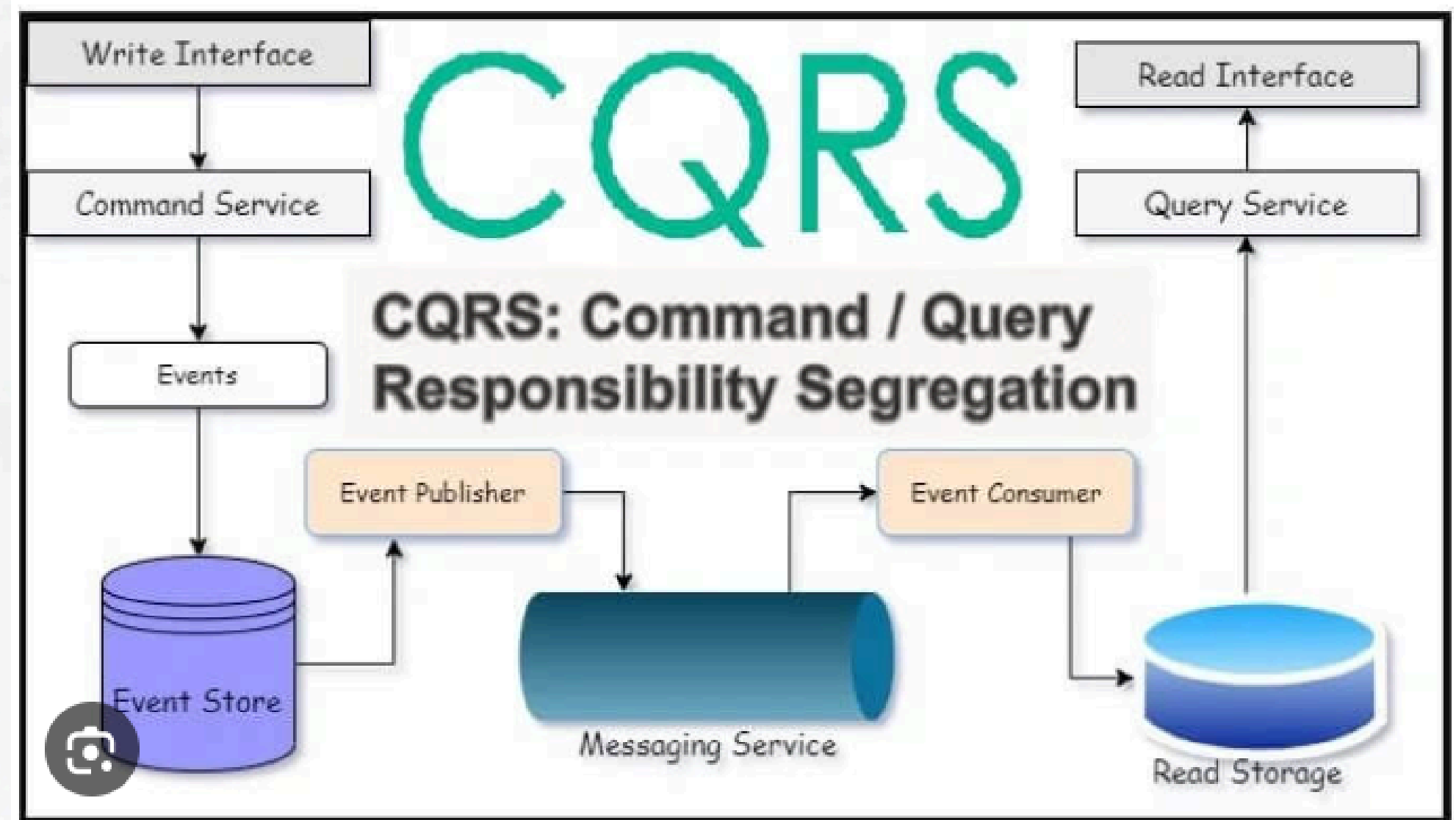


Tipos de patrones de software orientados a APIs

CQRS (Command Query Responsibility Segregation)

La Segregación de Responsabilidades de Comandos y Consultas (CQRS) es un patrón de diseño que separa las operaciones de escritura de datos (comandos) de las de lectura (consultas). Esta segregación permite optimizar, escalar y gestionar de forma independiente las partes de lectura y escritura de una aplicación.

CQRS se utiliza en aplicaciones de alto rendimiento. Al separar las consultas de los comandos, podemos por ejemplo utilizar dos bases de datos, una NOSQL, ya que son más rápidas a la hora de recuperar información, y otra SQL para los comandos (inserción, actualización y borrado) que son más veloces que las NOSQL a la hora de realizar estas acciones..

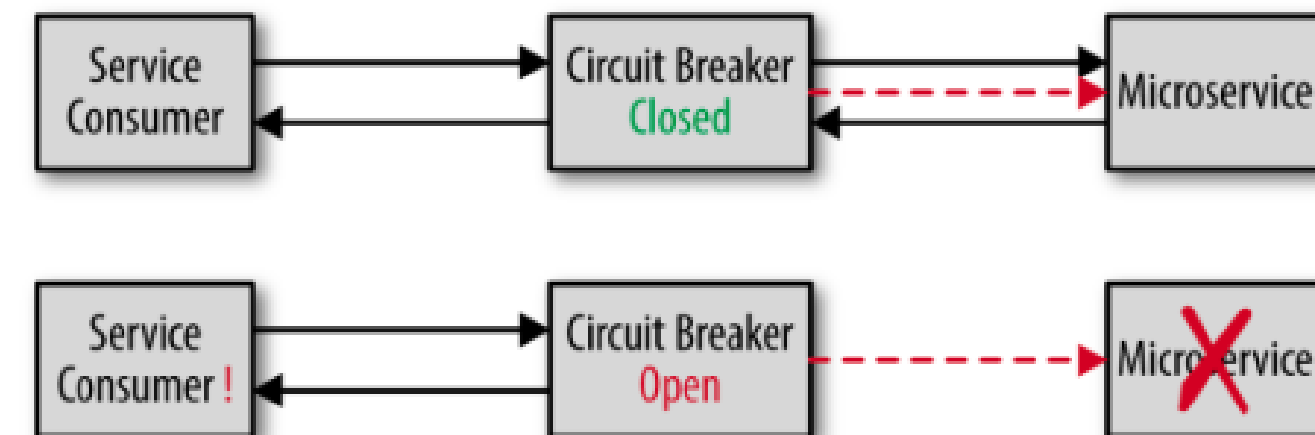


Tipos de patrones de software orientados a APIs

Circuit Breaker Pattern

Técnica para mejorar la resiliencia de los sistemas, especialmente en aplicaciones distribuidas y microservicios, que funciona como un interruptor automático de seguridad. Se usa para prevenir fallos en cascada al monitorear el estado de un servicio y, si detecta fallos repetidos, interrumpe temporalmente la comunicación con ese servicio para evitar agotar recursos y sobrecargar el sistema. El patrón opera en tres estados: Cerrado (la operación funciona), Abierto (la operación falla y se detiene), y Semiabierto (se permiten algunas pruebas para ver si el servicio se ha recuperado).

Circuit Breaker Pattern



BIBLIOGRAFÍA

- Barrón, J. P. G., Manso, M. Á., Alcarria, R., & Gomez, R. P. (2014, July). A mobile crowdsourcing platform for urban infrastructure maintenance. In 2014 Eighth International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing (pp. 358-363). IEEE.
- Yan, M., Sun, H., & Liu, X. (2014, November). iTest: testing software with mobile crowdsourcing. In Proceedings of the 1st International Workshop on Crowd-based Software Development Methods and Technologies (pp. 19-24).
- Kufner, J., & Mařík, R. (2019). Restful State Machines and SQL Database. IEEE Access, 7, 144603-144617.
- Guamán, D., Delgado, S., & Pérez, J. (2021). Classifying model-view-controller software applications using self-organizing maps. IEEE Access, 9, 45201-45229.
- Casciaro, M. (2020). Node.js Design Patterns. Packt Publishing Ltd.

BIBLIOGRAFÍA

- Gough, J., Bryant, D., & Auburn, M. (2023). Mastering API Architecture: Design, Operate, and Evolve API-Based Systems. O'Reilly Media, Inc.